



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程：操作系统原理实验

任课教师：刘宁

实验题目：从用户态到内核态

专业名称：计算机科学与技术（人工智能与大数据方向）

学生姓名：陶昕璨

学生学号：24325242

实验地点：实验中心 D501

实验时间：2026/6/1-2026/6/9

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
实验任务一.....	1
实验任务二.....	7
实验任务三.....	18
实验任务四.....	24
五、实验总结与心得体会.....	30
六、附录：参考资料清单.....	30

Section 1 实验概述

在本次实验中，首先简单讨论保护模式下的特权级的相关内容。特权级保护是保护模式的特点之一，通过特权级保护，我们区分了内核态和用户态，从而限制用户态的代码对特权指令的使用或对资源的访问等。但是，用户态的代码有时不得不使用一些特权指令，如输入 / 输出等。因此介绍了系统调用的概念和通过中断实现系统调用的方法。通过系统调用，我们可以实现从用户态到内核态转移，然后在内核态下执行特权指令，执行完成后返回到用户态。在实现了系统调用后，我们通过三个步骤创建进程。在这里，需要重点理解如何通过分页机制来实现进程之间的虚拟地址空间的隔离。最后，本次实验将介绍 `fork/wait/exit` 的一种简洁的实现思路。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++/C 程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE (64-bit)
 - 代码编辑环境：Vim (终端) / vscode
 - 代码编译工具：GCC、g++、nasm、ld (链接器)、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：实现系统调用的方法
- 实验任务 2：进程的创建和调度
- 实验任务 3：fork 的实现
- 实验任务 4：wait & exit 的实现

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

任务要求：实现系统调用的方法——复现指导书中“系统调用的实现”一节，并回答以下问题：

1. 请解释为什么需要使用寄存器来传递系统调用的参数，以及我们是如何在执行 `int 0x80` 前在栈中找到参数并放入寄存器的。

2. 请使用 `gdb` 来分析在我们调用了 `int 0x80` 后，系统的栈发生了怎样的变化？`esp` 的值和在 `setup.cpp` 中定义的变量 `tss` 有什么关系？此外还有哪些段寄存器发生了变化？变化后的内容是什么？
3. 请使用 `gdb` 来分析在进入 `asm_system_call_handler` 的那一刻，栈顶的地址是什么？栈中存放的内容是什么？为什么存放的是这些内容？
4. 请结合代码分析 `asm_system_call_handler` 是如何找到中断向量号 `index` 对应的函数的。
5. 请使用 `gdb` 来分析在 `asm_system_call_handler` 中执行 `iret` 后，哪些段寄存器发生了变化？变化后的内容是什么？这些内容来自于什么地方？

● 实验步骤：

Q1. 请解释为什么需要使用寄存器来传递系统调用的参数，以及我们是如何在执行 `INT 0x80` 前在栈中找到参数并放入寄存器的

A1:

①主要原因在于系统调用伴随着从用户态到内核态的特权级转移，这引发了硬件级别的栈切换

- a. 参数隔离：若使用栈传参，参数在用户态下被压入用户栈
- b. 执行中断，CPU 会自动从 `TSS` 中读取特权级 0 的栈信息并覆盖 `esp`，此时当前栈变成了内核栈
- c. 内核中执行的 C 语言函数在编译后，默认通过当前的 `esp/ebp` 寄存器去栈中寻找参数。由于此时的内核栈中并没有之前在用户态压入的数据，导致内核无法获取参数

②利用栈基址指针 `ebp` 来精准定位（相对寻址）并取出这些参数，然后放入寄存器（C 语言调用，栈传参，编译器会**从右往左**将参数压入用户栈）

其中，通用寄存器是程序员自己可以确定是否需要保存的

```
asm_system_call:
    push ebp ; 备份旧ebp
    mov ebp, esp

    ; 保存用户态需要的寄存器
    push ebx
    push ecx
    push edx
    push esi
    push edi
    push ds
    push es
    push fs
    push gs

    ; 从用户栈中利用ebp相对寻址取出参数写入指定寄存器
    mov eax, [ebp + 2 * 4]
    mov ebx, [ebp + 3 * 4]
    mov ecx, [ebp + 4 * 4]
    mov edx, [ebp + 5 * 4]
    mov esi, [ebp + 6 * 4]
    mov edi, [ebp + 7 * 4]

    ; 参数都在寄存器中，安全触发软中断
    int 0x80
```

Q2: 请使用 GDB 来分析在我们调用了 INT 0x80 后，系统的栈发生了怎样的变化？ESP 的值和在 SETUP.CPP 中定义的变量 TSS 有什么关系？此外还有哪些段寄存器发生了变化？变化后的内容是什么？

A2:

```
remote Thread 1 In: asm system call
esp      0x8048fa8      0x8048fa8
ss       0x3b         59
cs       0x2b         43
eflags   0x212        [ AF IF ]
ds       0x33         51
es       0x33         51
fs       0x33         51
---Type <return> to continue, or q <return> to quit---
```

```
remote Thread 1 In: asm system call      L153 PC: 0xc00226d3
0xc0033b40 <tss>:      0x00000000      0xc00256a0      0x00000010      0x00000000
00
0xc0033b50 <tss+16>:   0x00000000      0x00000000      0x00000000      0x00000000
P00
0xc0033b60 <tss+32>:   0x00000000      0x00000000      0x00000000      0x00000000
00
0xc0033b70 <tss+48>:   0x00000000      0x00000000      0x00000000      0x00000000
---Type <return> to continue, or q <return> to quit---
```

```
remote Thread 1 In: asm system call handler L89
esp      0xc0025688      0xc0025688 <PCB_SET+8168>
ss       0x10          16
cs       0x20          32
eflags   0x12          [ AF ]
ds       0x33          51
es       0x33          51
fs       0x33          51
---Type <return> to continue, or q <return> to quit---

(gdb) x/6xw $esp
0xc0025688 <PCB_SET+8168>:      0x00000033      0xc00226d5      0x0000002b
0x00000212
0xc0025698 <PCB_SET+8184>:      0x08048fa8      0x0000003b
(gdb)
```

①系统的栈（esp）变化与 TSS 的关系

- **实验现象：**中断前 `esp = 0x8048fa8`（用户栈），中断后 `esp = 0xc0025688`（内核栈）。
- **与 TSS 的关系：**
 - 中断触发时，CPU 硬件自动读取 `tss.esp0`（值为 `0xc00256a0`）作为新栈顶。
 - 随后，CPU 在该内核栈中自动压入 **5 个用户态寄存器**（`ss, esp, eflags, cs, eip`），共占用 20 字节（`0x14`）。
 - **数学关系：**

$$\text{tss.esp0} - 20 \text{ 字节} = 0xc00256a0 - 0x14 = 0xc0025688$$

② 内核栈数据具体内容（`x/6xw $esp` 验证）

从当前 `$esp`（低地址）向栈底（高地址）查看，内存数据对应了用户态断点现场：

- `+0x00 (0xc002568c) -> EIP (0xc00226d5)`：用户态 `int 0x80` 的下一条返回地址。
- `+0x04 (0xc0025690) -> CS (0x0000002b)`：用户态代码段选择子（`RPL=3`）。
- `+0x08 (0xc0025694) -> EFLAGS (0x00000212)`：用户态标志寄存器（`IF=1` 开中断）。
- `+0x0c (0xc0025698) -> ESP (0x08048fa8)`：中断前的用户态栈指针。

- +0x10 (0xc002569c) -> **SS** (0x0000003b): 用户态栈段选择子 (RPL=3)。

③ 段寄存器的变化及内容

段寄存器在中断触发瞬间分为**硬件自动加载**和**保持不变**两类：

段寄存器	变化前 (Ring 3)	变化后 (Ring 0)	变化原因与内容意义
cs	0x2b	0x20	硬件自动加载：切换为内核代码段（GDT 索引 4, RPL=0）。
ss	0x3b	0x10	硬件自动加载：切换为 tss.ss0 指定的内核栈段（GDT 索引 2, RPL=0）。
ds / es / fs	0x33	0x33	保持不变：x86 硬件不会自发切换通用数据段。此时刚进入中断入口，内核尚未执行手动重置代码，故依然维持用户态旧值。

Q3: 请使用 GDB 来分析在进入 `ASM_SYSTEM_CALL_HANDLER` 的那一刻，栈顶的地址是什么？栈中存放的内容是什么？为什么存放的是这些内容？

- **思路分析：**在特权级由用户态（RING 3）陷落到内核态（RING 0）的瞬间，CPU 硬件会自动执行切换堆栈与压栈保存现场的操作

A3:

```
esp      0xc002568c      0xc002568c <PCB_SET+8172>
(gdb) p/x tss.esp0
$1 = 0xc00256a0
(gdb) x/5xw $esp
0xc002568c <PCB_SET+8172>:      0xc00226d5      0x0000002b      0x00000212
0x08048fa8
0xc002569c <PCB_SET+8188>:      0x0000003b
(gdb)
```

① 在进入 `ASM_SYSTEM_CALL_HANDLER` 的那一刻，栈顶的地址是什么？

- 栈顶地址 `$ESP = 0xc002568c`。
- **数据逻辑分析：**
 - `TSS.ESP0 = 0xc00256a0`
 - 按照 x86 硬件设计，特权级由 RING 3 陷落到 RING 0 时，CPU 硬件会自动压入 5 个环境寄存器（SS, ESP, EFLAGS, CS, EIP），共占用 20 字节（0x14）。
 - **数学关系推导：**

$0xc00256a0$ (tss.esp0) $- 0x14$ (20字节) = **0xc002568c**

② 栈中存放的内容

存放的内容依次为：

1. $[0xc002568c] \rightarrow 0x000226d5$ ：用户态的 **EIP**（指向用户程序中 `INT 0x80` 的下一条返回地址）
2. $[0xc0025690] \rightarrow 0x0000002b$ ：用户态的 **CS**（低两位为 3，代表 RPL=3 的用户代码段）
3. $[0xc0025694] \rightarrow 0x00000212$ ：用户态的 **EFLAGS**（标志寄存器，含有 `IF=1` 开中断位）
4. $[0xc0025698] \rightarrow 0x08048fa8$ ：用户态的 **ESP**（中断发生前，用户程序原本的用户态栈顶指针）
5. $[0xc002569c] \rightarrow 0x0000003b$ ：用户态的 **SS**（低两位为 3，代表 RPL=3 的用户栈段）

③ 为什么存放的是这些内容？

纯硬件自发行为：

- **为了堆栈安全隔离**：用户态 RING 3 的栈是不安全的。进入内核态必须换上安全的内核栈（TSS.ESP0）。
- **为了能安全返回用户态**：一旦切换了堆栈，原先用户态的栈指针（旧 SS 和旧 ESP）就会丢失。因此 CPU 必须在第一时间把它们锁死压入内核栈。
- **为了保全执行流现场**：随后，CPU 必须把标志位（EFLAGS）以及当前运行的位置（CS、EIP）自动压栈备份。

Q4：请结合代码分析 `ASM_SYSTEM_CALL_HANDLER` 是如何找到中断向量号 `INDEX` 对应的函数的

A4：汇编层——`ASM_SYSTEM_CALL_HANDLER`

```
push eax ; 压入eax, 作为C++函数的参数 (系统调用号)

; 栈段会从tss中自动加载

mov eax, DATA_SELECTOR ; 使用内核的段选择子
mov ds, eax
mov es, eax

mov eax, VIDEO_SELECTOR
mov gs, eax

pop eax ; 获得函数index

; 参数压栈
push edi
push esi
push edx
push ecx
push ebx

sti ; 开启中断, 允许更高优先级的中断
call dword[system_call_table + eax * 4] ; 根据index, 在中断表中找到对应的函数
cli
```

Q5: 请使用 GDB 来分析在 ASM_SYSTEM_CALL_HANDLER 中执行 IRET 后, 哪些段寄存器发生了变化? 变化后的内容是什么? 这些内容来自于什么地方?

A5:

执行 IRET 前

```
remote Thread 1 In: asm system call handler
esp      0xc002568c      0xc002568c <PCB_SET+8172>
ss       0x10          16
cs       0x20          32
eflags   0x86          [ PF SF ]
ds       0x33          51
es       0x33          51
fs       0x33          51

(gdb) x/5xw $esp
0xc002568c <PCB_SET+8172>:      0xc00226d5      0x0000002b      0x00000212
0x08048fa8
0xc002569c <PCB_SET+8188>:      0x0000003b
(gdb)
```

执行 IRET 后

```
remote Thread 1 In: asm system call handler
esp      0x8048fa8      0x8048fa8
ss       0x3b          59
cs       0x2b          43
eflags   0x212         [ AF IF ]
ds       0x33          51
es       0x33          51
fs       0x33          51
---Type <return> to continue, or q <return> to quit---
```

①CS (代码段寄存器) 和 SS (栈段寄存器) + ESP (栈顶指针寄存器) 和 EFLAGS (标志寄存器)

②

- CS 的内容: 由内核态的 0x20 变成了用户态的 0x2B
- SS 的内容: 由内核态的 0x10 变成了用户态的 0x3B
- ESP 的内容: 由内核栈地址 0xc002568c 变成了用户栈地址 0x8048FA8。
- EFLAGS 的内容: 由 0x86 变成了 0x212 (状态位更新, 且 IF=1 重新开启了用户态下的硬件中断)

③来自于 IRET 执行前, 内核栈顶 (0xc002568c) 所弹出的 5 个双字 (共 20 字节) 数据并赋存寄存器

----- 实验任务 2 -----

任务要求: 进程的创建和调度——复现 “进程的实现”“进程的调度”“第一个进程”三节, 并回答以下问题:

1. 请结合代码分析我们是如何在线程的基础上创建进程的 PCB 的 (即分析进程创建的三个步骤)。

2. 在进程的 PCB 第一次被调度执行时，进程实际上并不是跳转到进程的第一条指令处，而是跳转到 `load process`。请结合代码逻辑和 `gdb` 来分析为什么 `asm_switch_thread` 在执行 `iret` 后会跳转到 `load process`。
3. 在跳转到 `load process` 后，我们巧妙地设置了 `ProcessStartStack` 的内容，然后在 `asm_start_process` 中跳转到进程第一条指令处执行。请结合代码逻辑和 `gdb` 来分析我们是如何设置 `ProcessStartStack` 的内容，从而使得我们能够在 `asm_start_process` 中实现内核态到用户态的转移，即从特权级 0 转移到特权级 3 下，并使用 `iret` 指令成功启动进程的。
4. 结合代码，分析在创建进程后，我们对 `ProgramManager::schedule` 作了哪些修改？这样做的目的是什么？
5. 在进程的创建过程中，我们存在如下语句：

```
cpp
运行
int ProgramManager::executeProcess(const char
*filename, int priority)
{
    //找到刚刚创建的 PCB
    PCB *process =
    ListItem2PCB(allPrograms.back(),tagInAllList);
}
```

正如教程中所提到，“..... 但是，这样做是存在风险的，我们应该通过 `pid` 来找到刚刚创建的 PCB.....” 现在，同学们需要编写一个 `ProgramManager` 的成员函数 `findProgramByPid`：

```
PCB *findProgramByPid(int pid);
```

并用上面这个函数替换指导书中提到的“存在风险的语句”，替换结果如下：

```
int ProgramManager::executeProcess(const char
*filename, int priority)
{
    //找到刚刚创建的 PCB
    PCB *process = findProgramByPid(pid);
}
```

自行测试通过后，说一说你的实现思路，并保存结果截图。

- 实验步骤：

Q1：请结合代码分析我们是如何在线程的基础上创建进程的 PCB 的（即分析进程创建的三个步骤）。

A1：executeProcess

```
// 在线程创建的基础上初步创建进程的PCB
int pid = executeThread((ThreadFunction)load_process,
| | | | | (void *)filename, filename, priority);
if (pid == -1)
{
    interruptManager.setInterruptStatus(status);
    return -1;
}

// 找到刚刚创建的PCB
PCB *process = ListItem2PCB(allPrograms.back(), tagInAllList);

// 创建进程的页目录表
process->pageDirectoryAddress = createProcessPageDirectory();
if (!process->pageDirectoryAddress)
{
    process->status = ProgramStatus::DEAD;
    interruptManager.setInterruptStatus(status);
    return -1;
}

// 创建进程的虚拟地址池
bool res = createUserVirtualPool(process);
```

①创建进程的 PCB：复用线程机制，但是把入口函数（ThreadFunction）设置为 **load_process**，而不是用户程序真正的入口 filename

这是为了把新进程伪装成了一个内核线程。它的任务就是让 CPU 第一次调度到它时，先进入内核态的 load_process，由这个函数在内核安全环境下伪造好用户态的现场，最后通过 iret 指令把执行流送进特权级 3

②初始化页目录表：复制内核共享空间（高 1GB）+页目录表自映射（最后一项）——内核可以通过 0xfffff000 这个固定虚拟地址直接访问和修改该进程的页表

```

int ProgramManager::createProcessPageDirectory()
{
    // 从内核地址池中分配一页存储用户进程的页目录表
    int vaddr = memoryManager.allocatePages(AddressPoolType::KERNEL, 1);
    if (!vaddr)
    {
        //printf("can not create page from kernel\n");
        return 0;
    }

    memset((char *)vaddr, 0, PAGE_SIZE);

    // 复制内核目录项到虚拟地址的高1GB
    int *src = (int *) (0xfffff000 + 0x300 * 4);
    int *dst = (int *) (vaddr + 0x300 * 4);
    for (int i = 0; i < 256; ++i)
    {
        dst[i] = src[i];
    }

    // 用户进程页目录表的最后一项指向用户进程页目录表本身
    ((int *)vaddr)[1023] = memoryManager.vaddr2paddr(vaddr) | 0x7;

    return vaddr;
}

```

③初始化用户虚拟地址池：内核线程没有自己独立的地址空间，它们统一使用内核虚拟地址池。而用户进程必须拥有

```

bool ProgramManager::createUserVirtualPool(PCB *process)
{
    //计算低 3GB 空间内，用户可分配的页数
    int sourcesCount = (0xc0000000 - USER_VADDR_START) / PAGE_SIZE;
    int bitmapLength = ceil(sourcesCount, 8);

    // 计算位图所占的页数
    int pageCount = ceil(bitmapLength, PAGE_SIZE);
    //从内核空间中，为该进程的虚拟地址池分配位图所需的内存
    int start = memoryManager.allocatePages(AddressPoolType::KERNEL, pageCount);

    if (!start)
    {
        return false;
    }

    //初始化 PCB 内部的专属虚拟地址池
    memset((char *)start, 0, PAGE_SIZE * pageCount);
    (process->userVirtual).initialize((char *)start, bitmapLength, USER_VADDR_START);

    return true;
}

```

Q2: 在进程的 PCB 第一次被调度执行时，进程实际上并不是跳转到进程的第一条指令处，而是跳转到 LOAD_PROCESS。请结合代码逻辑和 GDB 来分析为什么 ASM_SWITCH_THREAD 在执行 IRET 后会跳转到 LOAD_PROCESS

A2:

①创建进程时使用了函数套用，传入的不是真正的函数，而是 LOAD_PROCESS

```
// 在线程创建的基础上初步创建进程的PCB
int pid = executeThread((ThreadFunction)load_process,
                        (void *)filename, filename, priority);
```

②当调度器 PROGRAMMANAGER.SCHEDULE() 决定切换到这个新进程时，它会调用经典的汇编函数 ASM_SWITCH_THREAD(CUR, NEXT);

```
; void asm_switch_thread(PCB *cur, PCB *next);
asm_switch_thread:
    push ebp ; 保护现场
    push ebx
    push edi
    push esi

    mov eax, [esp + 5 * 4]
    mov [eax], esp ; 保存当前栈指针到PCB中，以便日后恢复

    ; 切换到新进程next的栈
    mov eax, [esp + 6 * 4]
    mov esp, [eax] ; 此时栈已经从cur栈切换到next栈

    ; 恢复现场
    pop esi
    pop edi
    pop ebx
    pop ebp

    sti
    ret ; ret一个正常的函数返回，直接把 load_process 的地址加载到了 eip 中，iret才是执行特权级降级
```

③GDB 验证

切换 ESP 指令之后

```
(gdb) x/5xw $esp
0xc0024640 <PCB_SET+4000>: 0x00000000 0x00000000 0x00000000
0x00000000
0xc0024650 <PCB_SET+4016>: 0xc0020a8d
```

- 前 4 个 0x00000000: 对应线程初始栈中预留的 4 个通用寄存器占位符 (esi, edi, ebx, ebp)
- 0xc0020a8d: 这是当前栈中紧接着要被当作函数返回地址读取的值

```

first_thread (arg=0x0) at ../src/kernel/setup.cpp:38
(gdb) info registers eip
eip                0xc0020a8d      0xc0020a8d <first_thread(void*)>
(gdb)

```

\$EIP 变成了 0xc0020a8d（此处对应的符号地址即为 LOAD_PROCESS）

```

int ProgramManager::executeProcess(const char *filename, int priority)
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();

    // 在线程创建的基础上初步创建进程的PCB
    int pid = executeThread((ThreadFunction)load_process,
        (void *)filename, filename, priority);
}

```

```

int ProgramManager::executeThread(ThreadFunction function, void *parameter, const char *name, int

```

进程第一次被调度时无法直接跳转到用户第一条指令。它必须先由 ASM_SWITCH_THREAD 尾部的 RET 指令，跳转到内核态的 LOAD_PROCESS 中。在 LOAD_PROCESS 里为用户态平坦模式重新伪造好更高级别的 PROCESSSTARTSTACK 后，才能最终利用 IRET 真正脱壳进入用户程序的第一条指令

Q3: 在跳转到 LOAD_PROCESS 后，我们巧妙地设置了 PROCESSSTARTSTACK 的内容，然后在 ASM_START_PROCESS 中跳转到进程第一条指令处执行。请结合代码逻辑和 GDB 来分析我们是如何设置 PROCESSSTARTSTACK 的内容，从而使得我们能够在 ASM_START_PROCESS 中实现内核态到用户态的转移，即从特权级 0 转移到特权级 3 下，并使用 IRET 指令成功启动进程的

A3:

①依次填充了通用寄存器（均为 0）和用户数据段选择子 USER_DATA_SELECTOR0x33

```

void load_process(const char *filename)
{
    interruptManager.disableInterrupt();

    PCB *process = programManager.running; //记录当前正在运行的进程
    ProcessStartStack *interruptStack =
        (ProcessStartStack *)((int)process + PAGE_SIZE - sizeof(ProcessStartStack));

    /// 初始化通用寄存器，全部设为0
    interruptStack->edi = 0;
    interruptStack->esi = 0;
    interruptStack->ebp = 0;
    interruptStack->esp_dummy = 0;
    interruptStack->ebx = 0;
    interruptStack->edx = 0;
    interruptStack->ecx = 0;
    interruptStack->eax = 0;
    interruptStack->gs = 0;

    //设置段选择子为用户数据段选择子
    interruptStack->fs = programManager.USER_DATA_SELECTOR;
    interruptStack->es = programManager.USER_DATA_SELECTOR;
    interruptStack->ds = programManager.USER_DATA_SELECTOR;
}

```

②5 个核心寄存器

```
//// 设置EIP为传入的文件名指针（实际上应该是入口地址）,这个寄存器是iret的返回地址
interruptStack->eip = (int)filename;
//// 设置代码段选择子为用户代码段选择子
interruptStack->cs = programManager.USER_CODE_SELECTOR; // 用户模式平坦模式
interruptStack->eflags = (0 << 12) | (1 << 9) | (1 << 1); // IOPL, IF = 1 开中断, MBS = 1 默认

//// 为用户栈分配一页内存
interruptStack->esp = memoryManager.allocatePages(AddressPoolType::USER, 1);
if (interruptStack->esp == 0)
{
    printf("can not build process!\n");
    process->status = ProgramStatus::DEAD;
    asm_halt();
}
//// 栈顶设置为分配页的末尾
interruptStack->esp += PAGE_SIZE;
// 设置栈段选择子为用户栈段选择子
interruptStack->ss = programManager.USER_STACK_SELECTOR;
```

- **interruptStack->ss = USER_STACK_SELECTOR (0x3b, RPL=3)**->对应用户态栈段
- **interruptStack->esp = USER_ESP** (通过 `allocatePages` 分配的用户栈顶)->对应用户态栈指针。
- **interruptStack->eflags = (0 << 12) | (1 << 9) | (1 << 1) (0x212)** -> 强行设置 **IOPL = 0**（剥夺用户态直接访问 IO 端口的特权），同时 **IF = 1** 保持开中断。
- **interruptStack->cs = USER_CODE_SELECTOR (0x2b, RPL=3)** -> 对应用户态代码段。
- **interruptStack->eip = (int)filename**（如 `first_process` 的函数指针） -> 对应用户程序的第一条指令地址

③调用

```
asm_start_process((int)interruptStack);
```

```
asm_start_process:
    jmp $
    mov eax, dword[esp+4]; 取出 ProcessStartStack 的起始地址
    mov esp, eax; 将当前的内核栈指针 ESP 挪到该结构体底部
    popad; 弹出 8 个通用寄存器值 (恢复为 0)
    ; 弹出恢复用户态数据段选择子 (0x33)
    pop gs;
    pop fs;
    pop es;
    pop ds;

    iret; 执行中断返回指令
```

此时 ESP 的地址是返回地址，参数的地址是 PROCESSSTACK 的地址，结合 PROCESSSTACK 的结构，弹出通用寄存器，段寄存器之后就获得 5 个关键的寄存器，此时就可以使用 IRET 正确跳转（IRET 弹出次序：EIP, CS, EFLAGS, ESP, SS）

④GDB

在 ASM_START_PROCESS 执行 IRET 前

```
remote Thread 1 In: asm_start_process L49
esp      0xc002568c      0xc002568c <PCB_SET+8172>
cs        0x20          32
ss        0x10          16
eflags    0x92          [ AF SF ]
ds        0x33          51
es        0x33          51
fs        0x33          51

(gdb) x/5xw $esp
0xc002568c <PCB_SET+8172>:      0xc0020a62      0x0000002b      0x00000202
0x00000000
0xc002569c <PCB_SET+8188>:      0x0000003b
```

执行 IRET 后

```
(gdb) info registers esp cs ss eflags eip
esp      0x8049000      0x8049000
cs        0x2b          43
ss        0x3b          59
eflags    0x202          [ IF ]
eip      0xc0020a62      0xc0020a62 <first_process()>
```

- cs（代码段）：由 0x20 变为 0x2b。代表当前处理器特权级（CPL）成功降为 Ring 3
- ss（栈段）：由 0x10 变为 0x3b。成功切到用户栈段。

- esp（栈顶指针）：由内核栈 0xc002568c 变为 0x8049000。处理器脱离了内核栈，使用独立的低 3GB 用户栈空间
- eip（程序计数器）

Q4：结合代码，分析在创建进程后，我们对 PROGRAMMANAGER::SCHEDULE 作了哪些修改？这样做的目的是什么？

A4：

①在原本的上下文切换（寄存器状态切换）之上，增加了“运行环境的切换”

核心修改增加了 ACTIVATEPROGRAMPAGE(NEXT) 函数

SCHEDULE 中

```
//激活/切换新进程的虚拟地址空间以及更新 TSS 栈
activateProgramPage(next);
//汇编上下文切换（切换寄存器、切内核栈）|
asm_switch_thread(cur, next);

interruptManager.setInterruptStatus(status);
```

ACTIVATEPROGRAMPAGE(NEXT)

```
void ProgramManager::activateProgramPage(PCB *program)
{
    int paddr = PAGE_DIRECTORY;

    /// 如果该 PCB 拥有独立的页目录表地址，说明它是一个“用户进程”，而非“内核线程”
    if (program->pageDirectoryAddress)
    {
        /// 更新 TSS 中的特权级 0 栈顶
        tss.esp0 = (int)program + PAGE_SIZE;
        /// 获取该进程页目录表的物理地址
        paddr = memoryManager.vaddr2paddr(program->pageDirectoryAddress);
    }
    //刷新 CR3 寄存器以切换页表
    asm_update_cr3(paddr);
}
```

切换页目录表、更新 TSS 内核栈，分别为了解决用户进程在 空间隔离 和 特权级转换 上的硬件硬性需求：

A. 切换页目录表（修改 CR3）的目的：实现进程虚拟地址空间隔离

- 根本目的：
 - 内核线程与用户进程共存：如果 NEXT 是内核线程（PAGE_DIRECTORYADDRESS 为 0），则将其 CR3 指向内核公共的页目录表（PAGE_DIRECTORY）

- **内存安全与彻底隔离**：由于不同进程的低 3GB (0 ~ 3GB) 虚拟地址映射到了完全不同的物理内存上，切换 CR3 意味着瞬间切换了整套 MMU 翻译机制

B. 更新 TSS 中 ESP0 的目的：解决 RING 3 陷落到 RING 0 时的“找栈”问题

- **根本目的**：
 - **多进程内核栈隔离**。
 - **锁定安全降落点**：x86 硬件规定，发生特权级提升时，CPU 必须自动读取 TR 寄存器指向的 TSS 结构体中的 SS0 和 ESP0 来换上内核栈。如果我们不在调度时动态将 TSS.ESP0 改为当前进程自己的 PCB 顶端，那么所有进程陷落时都会使用同一个内核栈

Q5：编写一个成员函数 FINDPROGRAMBYPID 替换存在风险的语句

A5：利用局部性原理 (PRINCIPLE OF LOCALITY) 从后往前遍历

核心逻辑：

- 获取当前全局程序队列 ALLPROGRAMS 的总长度 (SIZE)
- 开启一个从 SIZE - 1 开始递减到 0 的 FOR 循环。
- 在循环内部，利用 ALLPROGRAMS.AT(i) 读取对应的链表节点，并用宏/函数 LISTITEM2PCB 转换为 PCB* 结构体指针。
- 匹配 PID，一旦相等立刻返回该 PCB 指针；若循环结束仍未匹配，则返回 NULLPTR

替换锚点：

- 在 SRC/KERNEL/PROGRAM.CPP 中写入 FINDPROGRAMBYPID 成员函数。
- INCLUDE/PROGRAM.H 的 CLASS PROGRAMMANAGER 中添加该函数的公开声明。
- 修改 PROGRAMMANAGER::EXECUTEPROCESS 内部

```

PCB* ProgramManager::findProgramByPid(int pid){
    // 进入临界区：备份并关闭中断
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();

    int back = allPrograms.size() - 1;
    printf("size of the list is %d\n", back);
    for(int i = back; i >= 0; --i){
        if (ListItem2PCB(allPrograms.at(i), tagInAllList)->pid == pid){
            printf("index %d is what you need!\n", i);
            // 返回前恢复中断
            interruptManager.setInterruptStatus(status);
            return ListItem2PCB(allPrograms.at(i), tagInAllList);
        }
    }

    //返回前恢复中断
    interruptManager.setInterruptStatus(status);
    return nullptr;
}

```

由于在启动进程的过程中，设置了 EFLAGS 中的 IOPL 为 0。IOPL=0 表示只有特权级 0 下的代码才可以访问显存，而进程运行在特权级 3 下。但是在进程中又使用 PRINTF，所以会导致 QEMU 窗口发生闪退，这里使用 GDB 来观察函数运行结果

```

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0xC0010000
user pool
    start address: 0x4070000
    total pages: 15984 ( 62 MB )
    bit map start address: 0xC00107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0xC0010F9C
start process
size of the list is 1
checking index 1
index 1 is what you need!
size of the list is 2
checking index 2
index 2 is what you need!
size of the list is 3
checking index 3
index 3 is what you need!

```

实验任务 3

任务要求：fork 的实现——复现“fork”一小节的内容，并回答以下问题：

1. 请根据代码逻辑概括 fork 的实现的基本思路，并简要分析我们是如何解决“四个关键问题”的。
2. 请根据 gdb 来分析子进程第一次被调度执行时，即在 `asm_switch_thread` 切换到子进程的栈中时，`esp` 的地址是什么？栈中保存的内容是什么？
3. 从子进程第一次被调度执行时开始，逐步跟踪子进程的执行流程一直到子进程从 `fork` 返回，根据 gdb 来分析子进程的跳转地址、数据寄存器和段寄存器的变化。同时，比较上述过程和父进程执行完 `ProgramManager::fork` 后的返回过程的异同。
4. 请根据代码逻辑和 gdb 来解释子进程的 `fork` 返回值为什么是 0，而父进程的 `fork` 返回值是子进程的 `pid`。
5. 请解释在 `Programanager::schedule` 中，我们是如何从一个进程的虚拟地址空间切换到另外一个进程的虚拟地址空间的。

● 实验步骤：

Q1：根据代码逻辑概括 FORK 的实现的 basic 思路，并简要分析我们是如何解决“四个关键问题”的

A1：

①fork 实现的基本思路

- a) **进入内核**：进程在用户态调用 `fork()`，通过 `int 0x80` 陷落到内核态，CPU 自动将用户态现场压入该进程的 0 特权级栈（内核栈）。
- b) **骨架创建**：内核调用 `executeProcess` 分配一个新的 PCB（子进程），并为其构建基础的线程/进程运行栈
- c) **复制**：调用 `copyProcess`，将父进程内核栈里保存的用户态硬件上下文（`ProcessStartStack`）原封不动地复制给予进程，并强行将子进程的返回值（`EAX`）改为 0
- d) **资源复制**：复制虚拟地址池的位图，并在内核中转页（`Buffer`）的协助下，为子进程创建独立的页目录表和页表，将父进程物理页中的数据逐页拷贝到子进程的新物理页中。
- e) **双向返回**：子进程被唤醒后通过 `asm_start_process` 里的 `iret` 返回用

户态；父进程则直接从原系统调用路径返回，各自继续执行。

② “四个关键问题”

Q1：如何实现父子进程的代码段共享？

- 内核公共映射与函数复用
- 分析：用户进程的实体是用 C++ 函数，其编译后的机器码存放在内核高地址空间（3GB~4GB）。由于所有进程的页目录表在高地址空间都映射到了相同的内核页表，因此子进程天然与父进程共享这套完全相同的代码段

Q2：如何使得父子进程从相同的返回点开始执行？

- 克隆中断现场（EIP）与伪造伪调度栈。
- 分析：
 - 相同的用户态起点：父进程因 `int 0x80` 陷入内核时，其断点（下一条指令的 EIP——`asm_system_call_handler` 的返回地址）被保存在 `ProcessStartStack` 中。`copyProcess` 将这个包含 EIP 的现场完整复制给了子进程。
 - 子进程创建时，内核在其内核栈中伪造了 `asm_switch_thread` 的弹栈序列，将其首个执行点指向 `asm_start_process`。当子进程第一次被调度时，它会直接进入 `asm_start_process` 执行 `iret`，弹出刚刚克隆的 EIP，从而与父进程在相同的用户态位置（fork 返回点）醒来

Q3：除代码段外，进程包含的资源有哪些？

- 软件控制块：PCB 结构体本身（管理 PID、状态、优先级等）
- 内核栈：0 特权级栈（保存发生中断时的寄存器现场）
- 内存管理结构：虚拟地址池、页目录表和页表。
- 数据实体：用户态 3 特权级栈、堆区以及各类全局/局部数据所在的物理内存页

Q4：如何实现进程的资源在进程之间的复制？

- 软件层面的 `memcpy` + 硬件层面的“内核中转页”临时换页机制。
- 分析：对于 PCB 和内核栈等内核空间资源，直接在内核态下使用 `memcpy` 复制。对于用户空间资源（相同的虚拟地址，不同的物理地址），父进程

无法直接往子进程的虚拟地址写数据,内核通过分配一个内核空间的“中转页”)作为桥梁:

1. 在父进程空间下,将数据读入中转页;
2. 切换 CR3 寄存器进入子进程地址空间;
3. 将中转页的数据写到子进程对应的物理页中;
4. 切回父进程 CR3

Q2: 请根据 GDB 来分析子进程第一次被调度执行时,即在 ASM_SWITCH_THREAD 切换到子进程的栈中时,ESP 的地址是什么? 栈中保存的内容是什么?

A2:

子进程真实地址

```
(gdb) p/x child
$1 = 0xc0025de0

(gdb) info registers esp
esp             0xc0026d80             0xc0026d80 <PCB_SET+12192>
(gdb) x/16a $esp
0xc0026d80 <PCB_SET+12192>:  0x0      0x0      0x0      0x0
0xc0026d90 <PCB_SET+12208>:  0xc0022c20    0x0      0xc0026d9c    0x0
0xc0026da0 <PCB_SET+12224>:  0x0      0x8048fac  0xc0025dbc    0x0
0xc0026db0 <PCB_SET+12240>:  0x0      0x0      0x0      0x0
```

Q3: 从子进程第一次被调度执行时开始,逐步跟踪子进程的执行流程一直到子进程从 FORK 返回,根据 GDB 来分析子进程的跳转地址、数据寄存器和段寄存器的变化。同时,比较上述过程和父进程执行完 PROGRAMMANAGER::FORK 后的返回过程的异同

A3:

①子进程

跟踪 asm_switch_thread 的退出,进入 asm_start_process

```
(gdb) si
asm_start_process () at ../src/utils/asm_utils.asm:41
(gdb) info registers eip
eip             0xc0022c20             0xc0022c20 <asm_start_process>
```

查看数据寄存器(验证返回值)

```
(gdb) info registers eax ebx ecx edx esp
eax             0x0                    0
ebx             0x0                    0
ecx             0x0                    0
edx             0x0                    0
esp             0xc0026dcc             0xc0026dcc <PCB_SET+12268>
```

查看段寄存器（验证特权级）

```
(gdb) info registers cs ds es fs gs ss
cs             0x20      32
ds             0x33      51
es             0x33      51
fs             0x33      51
gs             0x0        0
ss             0x10      16
```

查看栈顶（验证 iret 弹出）

```
(gdb) x/5xw $esp
0xc0026dcc <PCB_SET+12268>: 0xc0022ccf 0x0000002b 0x00000216
0x08048f98
0xc0026ddc <PCB_SET+12284>: 0x0000003b
```

asm_start_process 执行完毕

```
(gdb) info registers eip
eip             0xc0022ccf 0xc0022ccf <asm_system_call+28>
(gdb) info registers cs ss
cs             0x2b      43
ss             0x3b      59
```

②父进程

```
(gdb) info registers eax ebx ecx edx esp
eax             0x2        2
ebx             0x0        0
ecx             0x0        0
edx             0x0        0
esp             0xc0025dcc 0xc0025dcc <PCB_SET+8172>
```

```
(gdb) x/5xw $esp
0xc0025dcc <PCB_SET+8172>: 0xc0022ccf 0x0000002b 0x00000216
0x08048f98
0xc0025ddc <PCB_SET+8188>: 0x0000003b
```

```
(gdb) info register cs ds es fs gs ss
cs             0x2b      43
ds             0x33      51
es             0x33      51
fs             0x33      51
gs             0x0        0
ss             0x3b      59
```

	父进程返回过程	子进程返回过程
数据寄存器 EAX （返回值）	EAX = 2 （真实的子进程 PID）	EAX = 0
退出前的特权级	处于内核态 Ring 0。段寄存器（如 cs=0x20）指示内核特权。	同样处于内核态 Ring 0（cs=0x20）。
退出后的特权级	降为用户态 Ring 3。段寄存器转换为用户段（cs=0x2b）。	同样降为用户态 Ring 3（cs=0x2b）。
用户态落脚点（EIP）	完全相同。父子进程 <code>iret</code> 弹出的用户态 EIP 均为 0x08048f98，即陷入中断前的下一条用户态指令。	
用户态数据栈（ESP）	完全相同。父子进程 <code>iret</code> 弹出的用户态 ESP 均为 0x0000003b。	

除了控制 fork 返回值，也就是 Pid 的 eax 寄存器不同之外，所有的寄存器都是一样的

Q4.请根据代码逻辑和 GDB 来解释子进程的 FORK 返回值为什么是 0，而父进程的 FORK 返回值是子进程的 PID

本质上是要看 EAX 寄存器在父子进程各自退出内核态时的最终数值

①代码底层逻辑

子进程：

```
bool ProgramManager::copyProcess(PCB *parent, PCB *child)
{
    // 复制进程0级栈
    ProcessStartStack *childpss =
        (ProcessStartStack *)((int)child + PAGE_SIZE - sizeof(ProcessStartStack));
    ProcessStartStack *parentpss =
        (ProcessStartStack *)((int)parent + PAGE_SIZE - sizeof(ProcessStartStack));
    memcpy(parentpss, childpss, sizeof(ProcessStartStack));
    // 设置子进程的返回值为0
    childpss->eax = 0;
}
```

父进程：父进程在退出 fork 时，EAX 自然而然承载的就是子进程的真实 pid

```

// 创建子进程
int pid = executeProcess("", 0);
if (pid == -1)
{
    interruptManager.setInterruptStatus(status);
    return -1;
}

// 初始化子进程
PCB *child = ListItem2PCB(this->allPrograms.back(), tagInAllList);
bool flag = copyProcess(parent, child); //复制父进程资源到子进程

if (!flag)
{
    child->status = ProgramStatus::DEAD;
    interruptManager.setInterruptStatus(status);
    return -1;
}

interruptManager.setInterruptStatus(status);
return pid;
}

```

②GDB

子进程返回 0

```

(gdb) si
asm_start_process () at ../src/utils/asm_utils.asm:49
(gdb) info registers eax
eax                0x0      0

```

父进程返回 PID

来到 fork 函数最后返回时

```

374
375         interruptManager.setInterruptStatus(status);
376         return pid;
377     }

```

退出 fork 、退出 syscall_fork ，来到系统调用总处理函数

<asm_system_call_handler> 的尾部

```

126         mov eax, [ASM_TEMP]
127
128         iret
129     asm_system_call:
130         push ebp

```

得到 eax 值

```

(gdb) si
(gdb) info registers eax
eax                0x2      2

```

Q5：请解释在 PROGRAMANAGER::SCHEDULE 中，我们是如何从一个进程的虚拟地址空间切换到另外一个进程的虚拟地址空间的。

A5：虚拟空间切换主要涉及页目录表的切换，在 schedule 中主要是通过 activateProgramPage 和 asm_update_cr3 来实现。主要是 asm_update_Cr3 中我们把下一个进程的页目录地址传入了 Cr3 寄存器，而这个寄存器内的地址是 cpu 分页机制查询的地址

- 软件层面（C++）：在调度时判断 next 是否为进程，并提取其 PCB 中准备好的页目录表物理地址
- 硬件层面（汇编）：通过内联汇编将该地址灌入 CR3 寄存器，引发 CPU 硬件页表的瞬间切换和 TLB 刷新

```
void ProgramManager::activateProgramPage(PCB *program)
{
    int paddr = PAGE_DIRECTORY;

    if (program->pageDirectoryAddress)
    {
        tss.esp0 = (int)program + PAGE_SIZE;
        // 获取新进程的页目录表物理地址
        paddr = memoryManager.vaddr2paddr(program->pageDirectoryAddress);
    }
    // 内联汇编将该地址写入 CR3 寄存器
    asm_update_cr3(paddr);
}
```

```
asm_update_cr3:
    push eax
    mov eax, dword[esp+8]
    mov cr3, eax
    pop eax
    ret
```

----- 实验任务 4 -----

任务要求：wait & exit 的实现——参考指导书中“wait”和“exit”两节的内容，实现 wait 函数和 exit 函数，回答以下问题：

1. 请结合代码逻辑和具体的实例来分析 exit 的执行过程。

2. 请解释进程退出后能够隐式调用 `exit` 的原因。（tips: 从栈的角度分析）
3. 请结合代码逻辑和具体的实例来分析 `wait` 的执行过程。
4. 如果一个父进程先于子进程退出，那么子进程在退出之前会被称为孤儿进程。子进程在退出后，从状态被标记为 `DEAD` 开始到被回收，子进程会被称为僵尸进程。请对代码做出修改，实现回收僵尸进程的有效方法。

● 实验步骤：

Q1: 请结合代码逻辑和具体的实例来分析 `EXIT` 的执行过程。

A1: `exit` 实际过程

- 标记 PCB 状态为 `DEAD` 并放入返回值。

```
void ProgramManager::exit(int ret)
{
    interruptManager.disableInterrupt();

    PCB *program = this->running;
    program->retValue = ret;
    program->status = ProgramStatus::DEAD;
```

- 如果 PCB 标识的是进程，则释放进程所占用的物理页、页表、页目录表和虚拟地址池 `bitmap` 的空间。否则不做处理。

```
void ProgramManager::exit(int ret)
{
    if (program->pageDirectoryAddress)
    {
        pageDir = (int *)program->pageDirectoryAddress;
        for (int i = 0; i < 768; ++i)
        {
            if (!(pageDir[i] & 0x1))
            {
                continue;
            }

            page = (int *) (0xffc00000 + (i << 12));

            for (int j = 0; j < 1024; ++j)
            {
                if (!(page[j] & 0x1))
                {
                    continue;
                }

                paddr = memoryManager.vaddr2paddr((i << 22) + (j << 12));
                memoryManager.releasePhysicalPages(AddressPoolType::USER, paddr, 1);
            }

            paddr = memoryManager.vaddr2paddr((int)page);
            memoryManager.releasePhysicalPages(AddressPoolType::USER, paddr, 1);
        }
    }
}
```

- 立即执行线程/进程调度

```
52
53     schedule();
```

Q2: 请解释进程退出后能够隐式调用 EXIT 的原因。（TIPS: 从栈的角度分析）

A2: 从栈的角度来说, main 函数的返回地址早就被 C 运行时库 (CRT) 硬编码指向了 __libc_start_main 中的 exit() 调用代码段

当 main 函数执行完毕清空自己的栈帧并执行 ret 回归时, 弹出的返回地址直接启动程序 exit

Q3: 请结合代码逻辑和具体的实例来分析 WAIT 的执行过程。

A3:

代码

①关闭中断

```
interrupt = interruptManager.getInterruptStatus();
interruptManager.disableInterrupt();
```

②遍历 allPrograms 寻找状态位 DEAD 的子进程

```
// 查找子进程
flag = true;
while (item)
{
    child = ListItem2PCB(item, tagInAllList);
    if (child->parentPid == this->running->pid)
    {
        flag = false;
        if (child->status == ProgramStatus::DEAD)
        {
            break;
        }
    }
    item = item->next;
}
```

③

情况 1: 找到 DEAD 子进程

回收 DEAD 子进程 PCB, 把子进程退出值写入父进程变量

```

if (item) // 找到一个可返回的子进程
{
    if (retval)
    {
        *retval = child->retValue;
    }

    int pid = child->pid;
    releasePCB(child);
    interruptManager.setInterruptStatus(interrupt);
    return pid;
}

```

情况 2：没有任何子进程

所有子进程都被回收 wait 返回-1

```

if (flag) // 子进程已经返回
{
    interruptManager.setInterruptStatus(interrupt);
    return -1;
}

```

情况 3：有子进程但子进程还没退出

```

else // 存在子进程，但子进程的状态不是DEAD
{
    interruptManager.setInterruptStatus(interrupt);
    schedule();
}

```

父进程让出 CPU，调度器选择其他进程或线程运行

实例

```

    bit map start address: 0x00010150
start process
thread exit
exit, pid: 3
exit, pid: 4
wait for a child process, pid: 3, return value: -123
wait for a child process, pid: 4, return value: 123934
all child process exit, programs: 2

```

当进程 2 调用 wait(&retval) 时，会通过系统调用进入 ProgramManager::wait()。首先遍历 allPrograms 链表，查找所有 parentPid == 2 的子进程。如果此时进程 3 和进程 4 都还没有执行 exit()，那么虽然能够找到子进程，但找不到状态为 DEAD 的子进程，因此调用 schedule() 让出 CPU，等待其他进程运行。之后假设进程 3 先执行 exit(-123)，其状态被设置

为 DEAD，退出值保存到 retValue 中。当进程 2 再次获得 CPU 后，会重新进入 wait() 并遍历 allPrograms，此时发现进程 3 满足 parentPid == 2 且 status == DEAD，于是读取其退出值 -123 写入 retval，记录其 PID 为 3，随后调用 releasePCB() 回收进程 3 的 PCB，并返回 PID 3。父进程继续调用 wait()，当进程 4 退出后，同样完成对进程 4 的回收并返回 PID 4。当进程 2 的所有子进程都被回收后，再次执行 wait() 时将找不到任何 parentPid == 2 的子进程，此时返回 -1，表示已经没有可等待的子进程。

Q4: 如果一个父进程先于子进程退出，那么子进程在退出之前会被称为孤儿进程。子进程在退出后，从状态被标记为 DEAD 开始到被回收，子进程会被称为僵尸进程。请对代码做出修改，实现回收僵尸进程的有效方法。

A4: 孤儿进程交给 1 号进程管理

①修改 void ProgramManager::exit(int ret)

```
program->status = ProgramStatus::DEAD;

// 将当前进程的子进程交给 1 号进程管理
ListItem *item = this->allPrograms.head.next;
PCB *child;

while (item)
{
    child = ListItem2PCB(item, tagInAllList);

    if (child->parentPid == program->pid)
    {
        child->parentPid = 1;
    }

    item = item->next;
}
```

②修改测试样例

```

src > kernel > setup.cpp > first_process()
void first_process()
{
    if (pid)
    {
        // 父进程 2 不 wait, 直接先退出
        printf("parent exit first, pid: %d\n", programManager.running->pid);
        exit(222);
    }
    else
    {
        // 子进程 4 延迟退出
        uint32 tmp = 0xffffffff;
        while (tmp)
            --tmp;

        printf("child exit, pid: %d, parentPid: %d\n",
            programManager.running->pid,
            programManager.running->parentPid);

        exit(444);
    }
}
else
{
    // 子进程 3 延迟退出
    uint32 tmp = 0xffffffff;
    while (tmp)
        --tmp;

    printf("child exit, pid: %d, parentPid: %d\n",
        programManager.running->pid,
        programManager.running->parentPid);

    exit(333);
}

```

```

void first_thread(void *arg)
{
    printf("start process\n");
    programManager.executeProcess((const char *)first_process, 1);
    //programManager.executeThread(second_thread, nullptr, "second", 1);

    int pid;
    int retval;

    while ((pid = wait(&retval)) != -1)
    {
        printf("init wait, pid: %d, retval: %d\n", pid, retval);
    }

    printf("init wait end, programs: %d\n", programManager.allPrograms.size());

    asm_halt();
}

```

此时可以看到进程 2 退出后，3 成为孤儿进程并且被进程 1 接管

```
bit map start address: 0xC0010F
start process
parent exit first, pid: 1
init wait, pid: 1, retval: 222
child exit, pid: 2, parentPid: 1
child exit, pid: 3, parentPid: 1
```

Section 5 实验总结与心得体会

实验总结

本次实验在前面线程管理和虚拟内存管理的基础上，实现了进程机制，并完成了进程创建、执行、退出以及回收的完整生命周期管理。

在进程创建部分，通过为进程建立独立的页目录表、用户虚拟地址空间和用户栈，实现了不同进程之间地址空间的隔离。同时，通过构造用户态运行环境并利用 `iret` 指令完成从 Ring0 到 Ring3 的切换，使用户程序能够在用户态独立运行。实验过程中进一步理解了用户态与内核态的区别，以及特权级切换的实现原理。

在进程复制部分，实现了 `fork` 系统调用。父进程通过复制 PCB、页表以及用户空间数据创建子进程，并利用内核栈和中断栈的修改实现父子进程返回值不同的效果。通过调试分析，理解了父子进程拥有相同代码和数据副本但具有不同 PID 的特点，也理解了操作系统如何通过上下文切换维持多个进程的并发执行。

在进程结束部分，实现了 `exit` 和 `wait` 系统调用。`exit` 负责释放进程占用的内存资源并保存退出信息，而 `wait` 负责由父进程回收已经结束的子进程 PCB，从而避免资源泄漏。

此外，在实验扩展部分，对孤儿进程和僵尸进程问题进行了处理。当父进程先于子进程退出时，将其子进程重新挂接到 1 号进程，由 1 号进程统一负责后续回收工作。

实验心得体会

通过本次实验，我对操作系统中的进程概念有了更加清晰的认识。在实际实现过程中，需要考虑 PCB 的组织方式、页表的建立与切换、用户态与内核态的转换以及父子进程之间的关系。本次实验不仅提高了我阅读和分析内核代码的能力，也让我对进程管理模块的整体设计有了更加系统的认识，为后续学习进程调度、同步互斥和文件系统等内容打下了基础。

Section 6 附录：参考资料清单

实验指导——[SYSU-2023-Spring-Operating-System: 中山大学 2023 学年春季操作系统课程 - Gitee.com](#)

实验参考——[OS lab8 从内核态到用户态 - RogersLuo](#)

大模型——[分页机制与内核高地址映射 - Google Gemini](#)